



UNIVERSIDAD NACIONAL AUTÓNOMA DE
MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN

GRÁFICA e INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 08

NOMBRE COMPLETO: Miranda González José Francisco

N° de Cuenta: 318222327

GRUPO DE LABORATORIO: 03

GRUPO DE TEORÍA: 04

SEMESTRE 2025-1

FECHA DE ENTREGA LÍMITE: 12/10/24

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

Reporte de práctica 8: Iluminación 2

Instrucciones:

1. Agregar un spotlight (que no sea luz de color blanco ni azul) que parta del cofre de su coche y al abrir y cerrar el cofre ilumine en esa dirección.
2. Agregar luz de tipo spotlight para el coche de tal forma que al avanzar (mover con teclado hacia dirección de X negativa) ilumine con un spotlight hacia adelante y al retroceder ((mover con teclado hacia dirección de X positiva) ilumine con un spotlight hacia atrás. Son dos spotlights diferentes que se prenderán y apagarán de acuerdo con alguna bandera asignada por ustedes.
3. Agregar otra luz de tipo puntual ligada a un modelo elegido por ustedes (no lámpara) y que puedan prender y apagar de forma independiente con teclado tanto la luz de la lámpara como la luz de este modelo (la luz de la lámpara debe de ser puntual, si la crearon spotlight en su reporte 7 tienen que cambiarla a luz puntual).

Actividades realizadas

Para completar las actividades solicitadas realice lo siguiente:

Para la actividad 1:

Entre los cambios nuevos dentro del archivo main.ccp (omitiendo la explicación de prácticas y ejercicios pasados).

Cree la spotlight para el cofre en la cual deje los valores de posición y dirección en 0, ya que estos se los heredaran después.

```
493 // COFRE <-----
494 spotLights[2] = SpotLight(
495     1.0f, 0.0f, 1.0f,
496     3.0f, 1.0f,
497     0.0f, 0.0f, 0.0f,
498     0.0f, 0.0f, 0.0f,
499     0.3f, 0.01f, 0.001f,
500     15.0f);
501
502 spotLightCount++;
```

Cree una matriz auxiliar para almacenar las transformaciones del cofre.

```

673 // PARA LA JERARQUIA
674 //////////////////////////////////////////////////
675 glm::mat4 modelaux(1.0); // COCHE
676 glm::mat4 modelauxH(1.0); // HELICOPTERO
677 glm::mat4 modelauxC(1.0); // COFRE
678 //////////////////////////////////////////////////

```

Después realice la jerarquía de la luz con el cofre.

```

747 model = modelaux; // A PARTIR DEL COCHE
748
749 // COFRE
750
751 model = glm::translate(model, glm::vec3(0.0f, 3.0f, 12.5f));
752 model = glm::rotate(model, glm::radians(mainWindow.getarticulacionCofre()), glm::vec3(1.0f, 0.0f, 0.0f)); // Rotar con: M y C
753
754 modelauxC = model; // GUARDAR
755
756 model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
757 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
758 /*color = glm::vec3(0.0f, 1.0f, 0.0f);
759 glUniform3fv(uniformColor, 1, glm::value_ptr(color));*/
760 Cofre.RenderModel();
761
762 model = modelauxC; // A PARTIR DEL COFRE
763
764 // PARA LA LUZ AZUL CON JERARQUIA
765
766 glm::vec3 posicionC = glm::vec3(model[3]);
767 glm::vec3 direccionC = glm::vec3(model[2]);
768
769 spotLights[2].SetFlash(posicionC, direccionC);

```

Para la actividad 2:

Cree el arreglo para los faros del coche.

```

123 // PARA PRACTICA 8
124 //////////////////////////////////////////////////
125 Spotlight spotlightFaro[MAX_SPOT_LIGHTS];
126 //////////////////////////////////////////////////

```

Cree las spotlight de los faros.

```

515 // PARA LOS FAROS DELNATEROS Y TRASEROS
516
517 unsigned int spotLightCountFaros = 0;
518
519 // ADELANTE <-----
520 spotlightFaro[0] = Spotlight(
521     1.0f, 0.5f, 0.0f,
522     3.0f, 1.0f,
523     0.0f, 0.0f, 0.0f,
524     0.0f, 0.0f, 0.0f,
525     0.3f, 0.01f, 0.001f,
526     15.0f);
527
528 spotLightCountFaros++;
529
530 // ATRAS <-----
531 spotlightFaro[1] = Spotlight(
532     0.5f, 1.0f, 0.5f,
533     3.0f, 1.0f,
534     0.0f, 0.0f, 0.0f,
535     0.0f, 0.0f, 0.0f,
536     0.3f, 0.01f, 0.001f,
537     15.0f);
538
539 spotLightCountFaros++;

```

Después establecí las condiciones necesarias para mostrar la spotlight delantera, trasera y las spotlights originales (las del ejercicio de clase pasado).

```
655 // U (ADELANTE) . Y (ATRAS)
656 // L (ORIGINAL)
657
658 if (mainWindow.gethaciaAdelanteAtras() == true && mainWindow.getinicio() != false)
659 {
660     shaderList[0].SetSpotLights(spotlightFaro, spotLightCountFaros - 1); // ADELANTE
661 }
662 if (mainWindow.gethaciaAdelanteAtras() == false && mainWindow.getinicio() != false)
663 {
664     shaderList[0].SetSpotLights(spotlightFaro + 1, spotLightCountFaros - 1); // ATRAS
665 }
666 if (mainWindow.getinicio() == false)
667 {
668     shaderList[0].SetSpotLights(spotLights, spotLightCount); // ORIGINAL
669 }
```

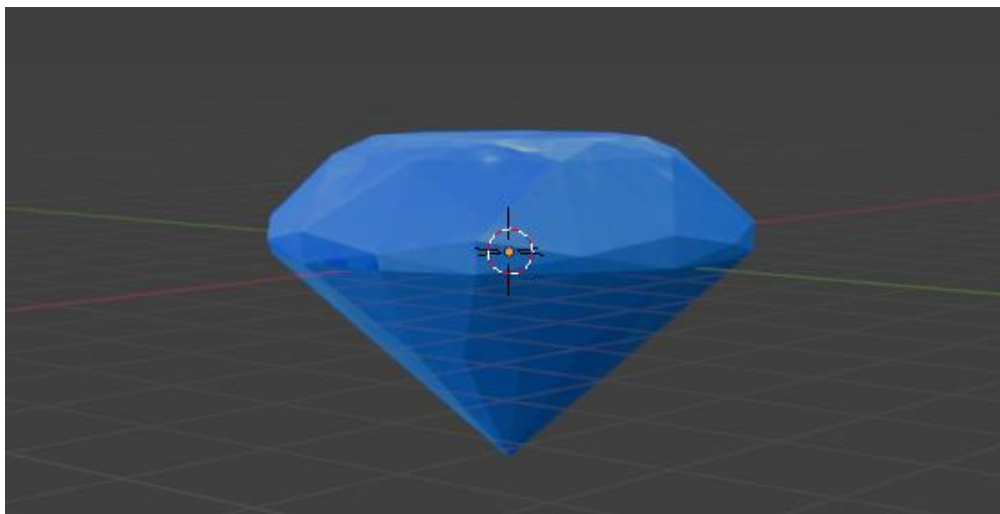
Utilizando la lógica anterior, elegí entre las dos spotlight que se iban a mostrar.

```
723 model = modelaux; // A PARTIR DEL COCHE
724
725 // PARA LA LUZ AZUL CON JERARQUIA
726
727 if (mainWindow.gethaciaAdelanteAtras() == true)
728 {
729     glm::vec3 posicion = glm::vec3(model[3]) + glm::vec3(-12.0f, -1.0f, 0.0f); // AJUSTAR LA POSICION A PARTIR DEL COCHE
730     // ESTO HACE QUE LA LUZ SE ACOMODE ENFRENTA DEL COCHE, PERO SI INTENTAMOS ROTAR EL COCHE LA LUZ ESTARA SEPAEADA DEL PUNTO DE ORIGEN, POR LO CUAL SE VERA DESFASADA
731     // COMO LA ROTACION DEL COCHE NO SE SOLICITA EN LA PRACTICA REALMENTE NO IMPORTA LO ANTERIOR
732     glm::vec3 direccion = glm::vec3(model[2]);
733     spotlightFaro[0].setFlash(posicion, direccion);
734 }
735
736 if (mainWindow.gethaciaAdelanteAtras() == false)
737 {
738     glm::vec3 posicion = glm::vec3(model[3]) + glm::vec3(10.0f, -1.0f, 0.0f);
739     glm::vec3 direccion = glm::vec3(model[2]);
740     direccion = - direccion;
741     spotlightFaro[1].setFlash(posicion, direccion);
742 }
743
744
745 }
```

Esto da la impresión de que solo se está cambiando entre las luces del arreglo sin apagarlas, pero las condiciones anteriores se encargan de que realmente las apague.

Para la actividad 3:

En blender exporte el modelo para esta actividad (diamante).



Cree la textura.

```
81 // PARA PRACTICA 8
82 //////////////////////////////////////////////////
83 Texture Diamante;
84 //////////////////////////////////////////////////
```

Cree el modelo.

```
101 // PARA PRACTICA 8
102 //////////////////////////////////////////////////
103 Model DiamanteM;
104 //////////////////////////////////////////////////
```

Cargue la textura.

```
384 // PARA PRACTICA 8
385 //////////////////////////////////////////////////
386 Diamante = Texture("Textures/Tex_0001_0.png");
387 Diamante.LoadTextureA();
388 //////////////////////////////////////////////////
```

Cargue el modelo.

```
415 // PARA PRACTICA 8
416 //////////////////////////////////////////////////
417 DiamanteM = Model();
418 DiamanteM.LoadModel("Models/Diamante.obj");
419 //////////////////////////////////////////////////
```

Cree su pointlight y la posición donde el modelo.

```
443 // POINTLIGHTS
444
445 //contador de luces puntuales
446 unsigned int pointLightCount = 0;
447
448 // POINTLIGHT DIAMANTE <-----
449 pointLights[0] = PointLight(
450     0.0f, 0.0f, 1.0f,
451     0.9f, 0.5f,
452     -60.0f, 3.5f, -30.0f,
453     0.0f, 0.1f, 0.06f);
454
455 pointLightCount++;
```

Después, al igual que con los faros establecí las diferentes condiciones que permitían prender y apagar las pointlight de los modelos de forma independiente.

Al igual que las condiciones de los faros, estas dependían de lo establecido en los archivos window.

```

610 // F (PRENDE DIAMANTE) . G (APAGA DIAMANTE)
611 // B (PRENDE LAMPARA) . N (APAGA LAMPARA)
612 // Q (ORIGINAL)
613
614 if (mainWindow.getinicio2() == false) // TODAS LAS LUCES APAGADAS
615 {
616     shaderList[0].SetPointLights(pointLights, pointLightCount - 2);
617 }
618 else if (mainWindow.getapagaPrende() == true &&
619     mainWindow.getapagarPrenderLampara() == false &&
620     mainWindow.getinicio2() != false) // SI QUIERO LA LUZ DEL DIAMANTE
621 {
622     shaderList[0].SetPointLights(pointLights, pointLightCount - 1);
623 }
624 else if (mainWindow.getapagaPrende() == false &&
625     mainWindow.getapagarPrenderLampara() == true &&
626     mainWindow.getinicio2() != false) // SI QUIERO LA LUZ DE LA LAMPARA
627 {
628     shaderList[0].SetPointLights(pointLights + 1, pointLightCount - 1);
629 }
630 else if (mainWindow.getapagaPrende() == true &&
631     mainWindow.getapagarPrenderLampara() == true &&
632     mainWindow.getinicio2() != false) // SI QUIERO AMBAS
633 {
634     shaderList[0].SetPointLights(pointLights, pointLightCount);
635 }
636 else if (mainWindow.getapagaPrende() == false &&
637     mainWindow.getapagarPrenderLampara() == false
638     && mainWindow.getinicio2() != false) // SI QUIERO APAGAGAR AMBAS
639 {
640     shaderList[0].SetPointLights(pointLights, pointLightCount - 2);
641 }
642 else if (mainWindow.getapagaPrende() == false &&
643     mainWindow.getapagarPrenderLampara() == true &&
644     mainWindow.getinicio2() != false) // SI QUIERO APAGAR LA LUZ DEL DIAMANTE
645 {
646     shaderList[0].SetPointLights(pointLights + 1, pointLightCount - 1);
647 }
648 else if (mainWindow.getapagaPrende() == true &&
649     mainWindow.getapagarPrenderLampara() == false &&
650     mainWindow.getinicio2() != false) // SI QUIERO APAGAR LA LUZ DE LA LAMPARA
651 {
652     shaderList[0].SetPointLights(pointLights, pointLightCount - 1);
653 }

```

En los archivos window realice los siguientes cambios.

Window.h.

```

27 // ACTIVIDADES PRACTICA 8
28 //////////////////////////////////////////////////
29 GLfloat gethaciaAdelanteAtras() { return haciaAdelanteAtras; }
30 GLfloat getinicio() { return inicio; }
31 GLfloat getapagaPrende() { return apagaPrende; }
32 GLfloat getinicio2() { return inicio2; }
33 //////////////////////////////////////////////////

```

```

62 // ACTIVIDADES PRACTICA 8
63 //////////////////////////////////////////////////
64 GLfloat haciaAdelanteAtras;
65 GLfloat inicio;
66 GLfloat apagaPrende;
67 GLfloat inicio2;
68 //////////////////////////////////////////////////

```

Window.cpp

```
28 // ACTIVIDADES PRACTICA 8
29 ///////////////////////////////////////////////////
30 haciaAdelanteAtras = false;
31 inicio = false;
32 apagaPrende = false;
33 inicio2 = false;
34 ///////////////////////////////////////////////////
```

Las condiciones fueron las siguientes:

Para los faros.

Con Y solo se mostrará el faro de enfrente, con U solo se mostrará el faro de atrás y con L regresamos a las spotlight originales.

```
128 // PARA LAS SPOTLIGHT
129 // ADELANTE
130 if (key == GLFW_KEY_Y)
131 {
132     theWindow->haciaAdelanteAtras = false;
133     theWindow->inicio = true;
134     theWindow-> muevex += 1.0;
135 }
136 // ATRAS
137 if (key == GLFW_KEY_U)
138 {
139     theWindow->haciaAdelanteAtras = true;
140     theWindow->inicio = true;
141     theWindow-> muevex -= 1.0;
142 }
143 // REINICIO
144 if (key == GLFW_KEY_L)
145 {
146     theWindow->inicio = false;
147 }
```

Para las pointlight.

Con B se prendera la lámpara y con N se apagará, con F se prendera el diamante y con G se apagará y con Q regresamos al estado original (todas apagadas).

```

148 // PARA LAS POINTLIGHT
149 // LAMPARA
150 if (key == GLFW_KEY_B)
151 {
152     theWindow->apagarPrenderLampara = true;
153     theWindow->inicio2 = true;
154 }
155 if (key == GLFW_KEY_N)
156 {
157     theWindow->apagarPrenderLampara = false;
158 }
159 // DIAMANTE
160 if (key == GLFW_KEY_F)
161 {
162     theWindow->apagaPrende = true;
163     theWindow->inicio2 = true;
164 }
165 if (key == GLFW_KEY_G)
166 {
167     theWindow->apagaPrende = false;
168 }
169 // REINICIO
170 if (key == GLFW_KEY_Q)
171 {
172     theWindow->inicio2 = false;
173     theWindow->apagarPrenderLampara = false;
174     theWindow->apagaPrende = false;
175 }

```

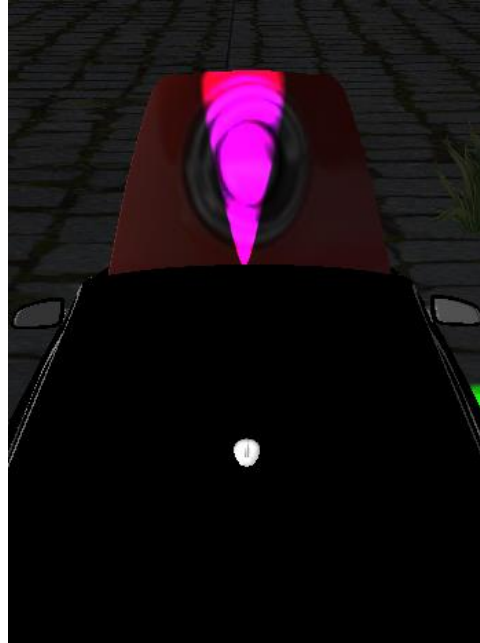
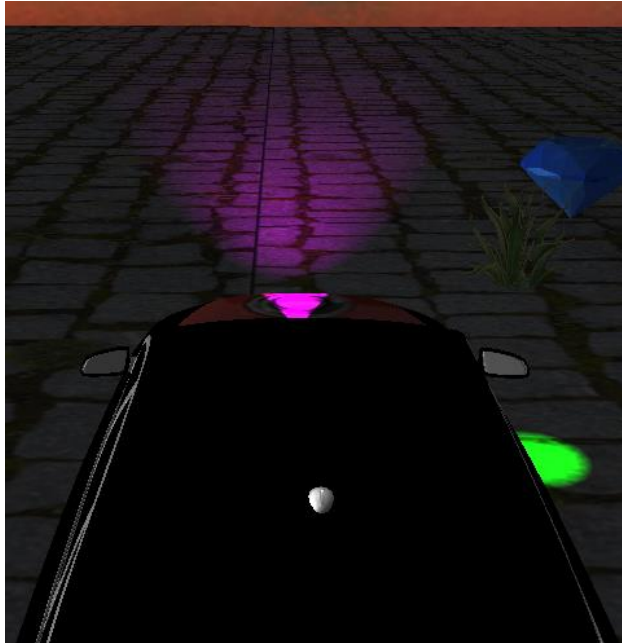
Ejecución del programa:

Vista inicial:



Actividad 1:

Cofre abajo y arriba.



Actividad 2:

Avanzar.



Retroceder.



Actividad 3:

Inicial.



Lampara prendida.



Diamante prendido.



Ambos prendidos.



Problemas presentados

1. Apagar y prender

Fue complicado establecer diferentes condiciones para prender y apagar las luces, sobre todo para las pointlight, porque no era como las spotlight que si prendía una apagaba la otra, en este caso si podían estar las dos prendidas y eso implicaba más condiciones.

Además de lo anterior, presente otro problema para decidir como apagar las luces.

En el laboratorio vimos que para apagar la última luz podíamos hacer lo siguiente:

```
shaderList[0].SetPointLights(pointLights, pointLightCount - 1);
```

Entonces busque información sobre cómo podía hacer para apagar la primera, ya que de lo contrario mi otra opción era crear un arreglo para cada luz e ir llamándolos según los necesitara, pero en ese caso no se estarían apagando las luces.

Entonces encontré que podía hacer lo siguiente:

```
shaderList[0].SetPointLights(pointLights + 1, pointLightCount - 1);
```

De esta forma no se estaría tomando en cuenta la primera luz.

No sé si esta solución haya sido la correcta, pero al momento de comprobar si estas se prendían y apagaban funcionaba todo de manera correcta.

Conclusión

Creo que lo más “complicado” de esta práctica fueron los últimos dos ejercicios, ya que tenía que pensar en qué condiciones establecer y como apagar las luces para que se mostrara el resultado esperado y eso me llevo mucho tiempo.

De estar correcta la solución propuesta para apagar y prender las luces me será de mucha utilidad a la hora de realizar futuras practicas y el proyecto.

Bibliografía

Studio Minus. (s.f). Colourpicker - Studio Minus. Consultado el 12 de octubre del 2024 de <https://www.studiominus.nl/colourpicker/>

The models resource. (s.f). Diamond. Consultado el 12 de octubre del 2024 de https://www.models-resource.com/pc_computer/legacyofdiscordfuriouswings/model/51333/